

BÀI THỰC HÀNH

LAB 01

Cơ bản về Digital Output

Điều khiển GPIO với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	Cấu trúc cơ bản của chương trình EduFramework	2
2.2	Hàm setup()	3
2.3	GPIO và Digital Output	3
2.4	Mức logic HIGH và LOW	3
2.5	LED Active-Low	4
3	Thiết lập phần cứng	4
3.1	Phần cứng sử dụng	4
3.2	Ánh xạ chân	4
4	EduFramework API	4
4.1	Tổng quan API	4
4.2	pinMode()	5
4.3	digitalWrite()	5
4.4	delay()	6
5	Bài thực hành	6
5.1	Yêu cầu bài toán	6
5.2	Tư duy thiết kế	6
5.3	Khung chương trình	7
6	Lời giải tham khảo và kiểm chứng	7
6.1	Mã nguồn tham khảo	7
6.2	Giải thích chương trình	8
6.3	Kết quả thực nghiệm	8
7	Mở rộng	8
8	Tài liệu tham khảo	9

1 Giới thiệu

1.1 Tổng quan bài lab

Digital Output là một trong những chức năng cơ bản nhất khi bắt đầu làm việc với vi điều khiển. Thông qua một chân GPIO được cấu hình ở chế độ output, chương trình có thể tạo ra các mức logic để điều khiển các phần tử phần cứng bên ngoài hoặc các phần tử tích hợp trực tiếp trên board.

Trong bài thực hành này, LED tích hợp trên MaaZEDU Development Board được sử dụng như một công cụ trực quan để minh họa cách cấu hình và điều khiển một Digital Output bằng EduFramework. Trọng tâm của bài lab không nằm ở thao tác nhấp nháy LED, mà nằm ở việc hiểu cấu trúc chương trình EduFramework, cách sử dụng Logical Pin và cách ứng dụng điều khiển một GPIO output.

Bài lab cũng giới thiệu cấu trúc chương trình C cơ bản được sử dụng xuyên suốt EduFramework Laboratory Series. Mỗi chương trình bắt đầu từ hàm `main()`, gọi `setup()` để thực hiện phần khởi tạo nền tảng trước khi cấu hình và chạy logic ứng dụng trong vòng lặp chính.

1.2 Mục tiêu

MỤC TIÊU

Sau khi hoàn thành bài lab này, người học có thể:

1. Trình bày được vai trò cơ bản của GPIO và Digital Output trong hệ thống nhúng.
2. Nhận biết được sự khác nhau giữa Logical Pin của EduFramework và chân MCU tương ứng.
3. Hiểu được cấu trúc cơ bản của một chương trình EduFramework viết bằng ngôn ngữ C.
4. Giải thích được vai trò của hàm `setup()` trong quá trình khởi tạo chương trình.
5. Sử dụng được `pinMode()` để cấu hình một chân digital ở chế độ output.
6. Sử dụng được `digitalWrite()` để thay đổi mức logic của Digital Output.
7. Sử dụng được `delay()` để tạo khoảng thời gian giữa các trạng thái của ứng dụng.

Ghi chú

Bản tài liệu này đang được sử dụng để kiểm thử cấu trúc và trình bày của hệ thống tài liệu EduFramework Laboratory Series. Nội dung lý thuyết của bản phát hành chính thức sẽ được đối chiếu với tài liệu kỹ thuật của NXP, tài liệu API và mã nguồn thực tế của EduFramework trước khi công bố.

2 Kiến thức nền

2.1 Cấu trúc cơ bản của chương trình EduFramework

Một ứng dụng EduFramework được viết bằng ngôn ngữ C và bắt đầu từ hàm `main()`. Khác với mô hình sketch sử dụng `setup()` và `loop()` như hai entry point độc lập, chương trình EduFramework vẫn sử dụng cấu trúc C truyền thống với một hàm `main()` duy nhất.

Trong `main()`, hàm `setup()` phải được gọi trước khi ứng dụng cấu hình hoặc sử dụng các chức năng khác của framework. Sau giai đoạn khởi tạo, chương trình có thể cấu hình các ngoại vi cần thiết và thực hiện logic ứng dụng trong vòng lặp `while (1)`.

Cấu trúc tổng quát có thể biểu diễn như sau:

```
#include "Arduino.h"

int main(void)
{
    setup();

    /* Application initialization */

    while (1)
    {
        /* Application logic */
    }

    return 0;
}
```

Mã nguồn 1.2.0. Cấu trúc cơ bản của một chương trình EduFramework

2.2 Hàm setup()

Trong các bài lab của EduFramework, `setup()` được gọi một lần ở đầu hàm `main()` trước khi các API khác được sử dụng. Vai trò chi tiết của hàm này sẽ được mô tả dựa trên mã nguồn thực tế của framework trong phiên bản tài liệu chính thức.

Ở góc nhìn của người sử dụng framework, quy tắc quan trọng trong bài lab đầu tiên là: **gọi `setup()` trước khi thực hiện phần cấu hình và logic ứng dụng.**

CẢNH BÁO

Không bỏ qua lời gọi `setup()` trong cấu trúc chương trình EduFramework. Các bài lab sau sẽ mặc định rằng phần khởi tạo nền tảng đã được thực hiện trước khi các API ngoại vi được sử dụng.

2.3 GPIO và Digital Output

General-Purpose Input/Output, thường được viết tắt là GPIO, là cơ chế cơ bản cho phép vi điều khiển tương tác với các tín hiệu số. Tùy theo cấu hình, một chân GPIO có thể được sử dụng để nhận trạng thái từ bên ngoài hoặc tạo ra một mức logic để điều khiển phần cứng.

Trong bài thực hành này, chân điều khiển LED được sử dụng ở chế độ Digital Output. Chương trình sẽ thay đổi trạng thái logic của chân này để tạo ra hai trạng thái phần cứng có thể quan sát trực tiếp.

Các đặc điểm cụ thể về cấu trúc GPIO, thanh ghi, điện áp và giới hạn điện của S32K144 cần được đối chiếu với tài liệu kỹ thuật chính thức của NXP [1], [2].

2.4 Mức logic HIGH và LOW

Ở tầng ứng dụng, EduFramework sử dụng các giá trị logic như `HIGH` và `LOW` để biểu diễn trạng thái của Digital Output. Cách biểu diễn này giúp mã nguồn ứng dụng dễ đọc hơn so với việc thao tác trực tiếp trên thanh ghi phần cứng.

Tuy nhiên, mức logic phần mềm không nhất thiết đồng nghĩa trực tiếp với trạng thái hiển thị của thiết bị. Cách phần cứng được mắc quyết định việc `HIGH` hay `LOW` tương ứng với trạng thái bật của thiết bị.

Khái niệm `HIGH` và `LOW` cũng được sử dụng trong mô hình Digital I/O của Arduino [4].

2.5 LED Active-Low

LED tích hợp trên board có thể được thiết kế theo cơ chế active-high hoặc active-low. Với phần cứng active-low, thiết bị được kích hoạt khi tín hiệu điều khiển ở mức logic thấp.

Trong bài lab chính thức, trạng thái điện của LED trên MaaZEDU sẽ được xác nhận lại từ schematic hoặc board documentation trước khi đưa ra kết luận cuối cùng.

Ghi chú

Việc dùng LED chỉ là phương tiện trực quan để quan sát Digital Output. Kiến thức cốt lõi của bài lab là cấu hình GPIO output và điều khiển mức logic của chân thông qua API EduFramework.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài thực hành sử dụng phần cứng tối thiểu để người học có thể tập trung vào cấu trúc chương trình và Digital Output.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144 và LED tích hợp phục vụ cho bài thực hành.
USB Cable	Kết nối board với hệ thống phát triển và cung cấp kết nối cần thiết trong quá trình làm bài lab.
J-Link Debug Probe	Thiết bị hỗ trợ nạp và debug chương trình trên S32K144.

Bảng 1.3.0. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân

EduFramework cung cấp các Logical Pin để ứng dụng không cần sử dụng trực tiếp tên port/pin vật lý của MCU trong mã nguồn mức cao.

Thành phần	Logical Pin	MCU Pin	Chức năng
LED đỏ tích hợp	LED_RED	PTD15	Digital Output

Bảng 1.3.1. Ánh xạ chân LED sử dụng trong bài thực hành

Trong mã nguồn ứng dụng, `LED_RED` được sử dụng như Logical Pin của EduFramework. Phần ánh xạ tới `PTD15` giúp liên hệ abstraction của framework với chân GPIO thực tế trên S32K144.

4 EduFramework API

4.1 Tổng quan API

Các API cần thiết cho bài thực hành Digital Output được tổng hợp trong bảng dưới đây. Phần mô tả chi tiết chỉ tập trung vào những thông tin cần thiết để thực hiện bài lab.

API	Mô tả
<code>pinMode(pin, mode)</code>	Cấu hình chế độ hoạt động của một Logical Pin digital.

API	Mô tả
<code>digitalWrite(pin, value)</code>	Thiết lập mức logic đầu ra cho một chân digital đã được cấu hình.
<code>delay(ms)</code>	Tạo khoảng thời gian chờ theo đơn vị millisecond.

Bảng 1.4.0. Các API EduFramework sử dụng trong bài thực hành

4.2 pinMode()

`pinMode()` được sử dụng để cấu hình chế độ hoạt động của một chân digital trước khi chân đó được sử dụng trong logic ứng dụng.

pinMode

Cấu hình chế độ hoạt động cho Logical Pin được chỉ định.

Cú pháp

```
pinMode(pin, mode);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>pin</code>	Logical Pin	Chân digital cần cấu hình.
<code>mode</code>	<code>OUTPUT</code>	Chế độ hoạt động cần thiết cho bài thực hành Digital Output.

Giá trị trả về

Không trả về giá trị.

Thông tin chính xác về kiểu dữ liệu và toàn bộ tập giá trị được hỗ trợ của từng tham số sẽ được lấy trực tiếp từ header/source EduFramework khi viết bản Lab 01 chính thức. Mô hình sử dụng API có thể được đối chiếu thêm với tài liệu `pinMode()` của Arduino [3].

4.3 digitalWrite()

`digitalWrite()` thay đổi mức logic của một chân digital output.

digitalWrite

Thiết lập mức logic đầu ra cho Logical Pin đã được cấu hình ở chế độ Digital Output.

Cú pháp

```
digitalWrite(pin, value);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>pin</code>	Logical Pin	Chân output cần điều khiển.
<code>value</code>	<code>HIGH</code> / <code>LOW</code>	Mức logic cần thiết lập cho chân output.

Giá trị trả về

Không trả về giá trị.

Tài liệu chính thức sẽ đối chiếu định nghĩa, kiểu tham số và hành vi API với mã nguồn EduFramework thay vì suy luận từ API Arduino [4].

4.4 delay()

`delay()` được sử dụng trong bài lab để giữ một trạng thái của ứng dụng trong một khoảng thời gian có thể quan sát được.

delay

Tạo khoảng thời gian chờ cho luồng thực thi chương trình.

Cú pháp

```
delay(ms);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
ms	millisecond	Khoảng thời gian chờ yêu cầu.

Giá trị trả về

Không trả về giá trị.

Trong các bài lab phức tạp hơn, cách tạo thời gian không blocking có thể được giới thiệu riêng. Ở bài đầu tiên, `delay()` được sử dụng để giữ ví dụ trực quan và tập trung vào Digital Output [5].

5 Bài thực hành

5.1 Yêu cầu bài toán

Hãy xây dựng một chương trình EduFramework điều khiển LED đảo tích hợp trên MaaZEDU Development Board thay đổi trạng thái theo chu kỳ.

Chương trình cần đáp ứng các yêu cầu sau:

- sử dụng cấu trúc chương trình C với `main()` ;
- gọi `setup()` trước khi sử dụng các API của EduFramework;
- sử dụng Logical Pin của framework để truy cập LED;
- cấu hình chân LED ở chế độ Digital Output;
- thay đổi trạng thái LED giữa hai mức logic;
- duy trì mỗi trạng thái trong khoảng 500 ms;
- lặp lại hành vi liên tục trong quá trình chương trình hoạt động.

Ở bước này, tài liệu chưa cung cấp lời giải hoàn chỉnh. Người học cần sử dụng kiến thức và thông tin API đã được trình bày ở các phần trước để xây dựng chương trình.

5.2 Tư duy thiết kế

Trước khi viết mã nguồn, có thể phân tích bài toán theo các bước tư duy sau:

Bước 1. Xác định tài nguyên cần điều khiển

Xác định Logical Pin đại diện cho LED tích hợp và liên hệ Logical Pin đó với phần cứng thực tế trên board.

Bước 2. Xác định bước khởi tạo chương trình

Xác định thứ tự gọi `setup()` và cấu hình Digital Output trước khi chương trình bắt đầu thay đổi mức logic của LED.

Bước 3. Xác định hai trạng thái đầu ra

Xác định hai mức logic cần được sử dụng để tạo ra hai trạng thái quan sát được của LED.

Bước 4. Xác định khoảng thời gian

Xác định vị trí cần áp dụng khoảng chờ 500 ms để mỗi trạng thái được giữ đủ lâu trước khi chuyển sang trạng thái tiếp theo.

Bước 5. Tổ chức vòng lặp chính

Tổ chức logic trong `while (1)` để chuỗi trạng thái được lặp lại liên tục trong suốt thời gian chương trình hoạt động. Các bước trên chỉ mô tả hướng tư duy. Người học vẫn cần tự chuyển chúng thành mã nguồn hoàn chỉnh.

5.3 Khung chương trình

Hoàn thiện phần còn thiếu trong chương trình sau:

```
#include "Arduino.h"

int main(void)
{
    setup();

    /* Configure the digital output here. */

    while (1)
    {
        /* Implement the required output sequence here. */
    }

    return 0;
}
```

Mã nguồn 1.5.0. Khung chương trình cho bài thực hành Digital Output

Hãy kiểm tra kết quả trên phần cứng trước khi xem phần lời giải tham khảo.

6 Lời giải tham khảo và kiểm chứng

6.1 Mã nguồn tham khảo

Một cách triển khai đáp ứng yêu cầu của bài thực hành được trình bày dưới đây.

```
#include "Arduino.h"

int main(void)
{
    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        digitalWrite(LED_RED, LOW);
        delay(500U);

        digitalWrite(LED_RED, HIGH);
        delay(500U);
    }

    return 0;
}
```

Mã nguồn 1.6.0. Mã nguồn tham khảo cho ứng dụng Digital Output

6.2 Giải thích chương trình

Chương trình bắt đầu từ `main()` và gọi `setup()` trước khi thực hiện phần cấu hình của ứng dụng. Sau đó, `pinMode()` cấu hình `LED_RED` thành Digital Output.

Trong vòng lặp `while (1)`, chương trình lần lượt ghi hai mức logic khác nhau ra LED bằng `digitalWrite()`.

Sau mỗi lần thay đổi trạng thái, `delay(500U)` giữ trạng thái hiện tại trong khoảng thời gian đủ để quan sát.

Khi kết thúc lần thực thi cuối của vòng lặp, chương trình quay lại đầu `while (1)` và tiếp tục chuỗi hoạt động.

Vì vậy trạng thái LED được thay đổi tuần hoàn trong suốt thời gian ứng dụng hoạt động.

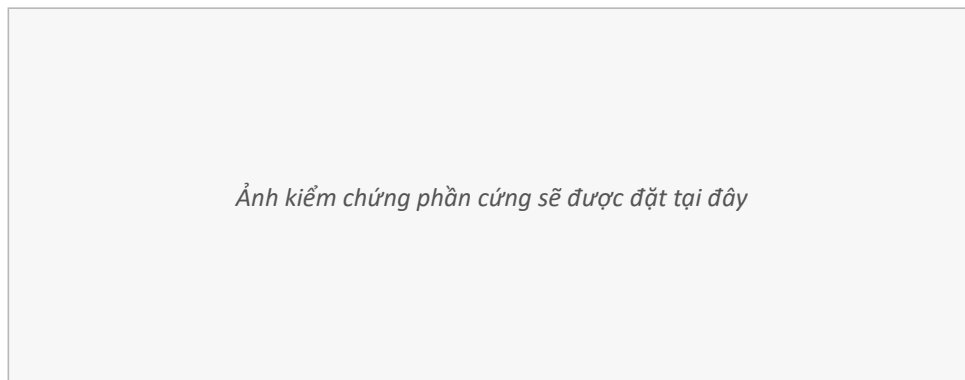
Phần mô tả chính thức sẽ xác nhận trạng thái `LOW / HIGH` tương ứng với LED bật hay tắt dựa trên schematic và tài liệu MaaZEDU thay vì chỉ dựa vào quan sát.

6.3 Kết quả thực nghiệm

KẾT QUẢ MONG ĐỢI

Sau khi chương trình được nạp và chạy thành công, LED tích hợp phải thay đổi trạng thái theo chu kỳ tương ứng với khoảng thời gian đã được cấu hình trong chương trình.

Bản phát hành chính thức sẽ sử dụng ảnh chụp thực tế trên MaaZEDU Development Board để làm bằng chứng kiểm chứng phần cứng.



Ảnh kiểm chứng phần cứng sẽ được đặt tại đây

Hình 1.6.0. Kết quả thực nghiệm của ứng dụng Digital Output

7 Mở rộng

Sau khi hoàn thành bài thực hành cơ bản, người học có thể tiếp tục thay đổi chương trình để quan sát và phân tích thêm hành vi của Digital Output.

Các hướng mở rộng gợi ý:

- Thay đổi khoảng thời gian giữa hai trạng thái của LED và quan sát sự thay đổi của chu kỳ.
- Sử dụng hai khoảng thời gian khác nhau cho trạng thái bật và tắt.
- Thay `LED_RED` bằng một LED tích hợp khác được EduFramework hỗ trợ.
- Điều khiển nhiều LED theo một chuỗi trạng thái xác định.
- Thêm một hoạt động khác vào vòng lặp chính và quan sát ảnh hưởng của `delay()` tới luồng thực thi chương trình.

Phần mở rộng không cung cấp lời giải tham khảo nhằm khuyến khích người học tự thử nghiệm và phát triển chương trình từ kiến thức đã học.

8 Tài liệu tham khảo

Danh sách dưới đây chỉ được sử dụng để kiểm tra bố cục reference của template. Khi viết bài Lab 01 chính thức, tên tài liệu, revision, metadata và URL sẽ được xác nhận từ nguồn chính thức trước khi đưa vào tài liệu.

- [1] NXP Semiconductors [,]
S32K1xx Series Reference Manual
[,] S32K1xx Series Reference Manual
[,] NXP Semiconductors
[.]
Truy cập trực tuyến
[.]

- [2] NXP Semiconductors [,]
S32K1xx Data Sheet
[,] NXP Semiconductors
[.]
Truy cập trực tuyến
[.]

- [3] Arduino [,]
pinMode()
[,] Arduino Language Reference
[.]
Truy cập trực tuyến
[.]

Truy cập ngày: 31/08/2026.

- [4] Arduino [,]
digitalWrite()
[,] Arduino Language Reference
[.]
Truy cập trực tuyến
[.]

Truy cập ngày: 31/08/2026.

- [5] Arduino [,]
delay()
[,] Arduino Language Reference
[.]
Truy cập trực tuyến
[.]

Truy cập ngày: 31/08/2026.